

Contest → end of Inter. module

1.5 hours contest

9pm to 10:30pm 10:30 - 11:30pm

Contest Contest
↪ 3 Questions discussion.

2 question
forming
cut off.

Contest → Friday

↳

Saturday reattempt.

Given a string **s** of lowercase characters, return the **count of pairs (i,j)** such that $i < j$ and **s[i]** is 'a' and **s[j]** is 'g'.

String s = 'a b e g a g' Ans $\Rightarrow$ 9.

String s = 'a c g d g a g' Ans $\Rightarrow$ 4.

String s = 'b c a g g a a g' Ans $\Rightarrow$ 5.

1) Brute force :-

String s = 'a b e g a g' n = 6.

int ans = 0;

s = a a a a a

for (i = 0; i < n; i++) {

T.C $\Rightarrow O(n^2)$

S.C $\Rightarrow O(1)$

if (s[i] == 'a') {

for (j = i+1; j < n; j++) {

if (s[j] == 'g') {

ans++;

}

}

}

}

return ans;

Optimized Solution :-

String s = 'a⁰b¹e²g³a⁴g⁵'

Observations :-

- For every 'g', we need to know the count of 'a' on left side of 'g'.
- We will store the count of 'a' and whenever 'g' is encountered, we will add the count of 'a' to the result.

Example :-

	a	c	b	a	g	k	a	g	g
Count a = 0	1	1	1	2	2	2	3	3	3
Ans = 0	0	0	0	0	2	2	2	5	8

count a = 0, ans = 0;

for (i = 0; i < n; i++) {

 if (s[i] == 'a') {

 count a++;

 } else if (s[i] == 'g') {

 ans += count a;

 }

return ans;

carry forward :-

T.C → O(n)

S.C → O(1).

← Subarrays →

A subarray is a contiguous part of an array. It is formed by selecting a range of elements from the array. A subarray can have one or more elements and must be a contiguous part of the original array.

arr[] → 4, 1, 2, 3, -1, 6, 9, 8, 12

(2, 3, -1, 6) ✓

(9) ✓

4, 1, 2, 3, -1, 6, 9, 8, 12 ✓

(4, 12) ✗

(1, 2, 6) ✗

(3, 2, 1, 4) ✗

Quiz

A[] = { 2, 4, 1, 6, -3, 7, 8, 4 }

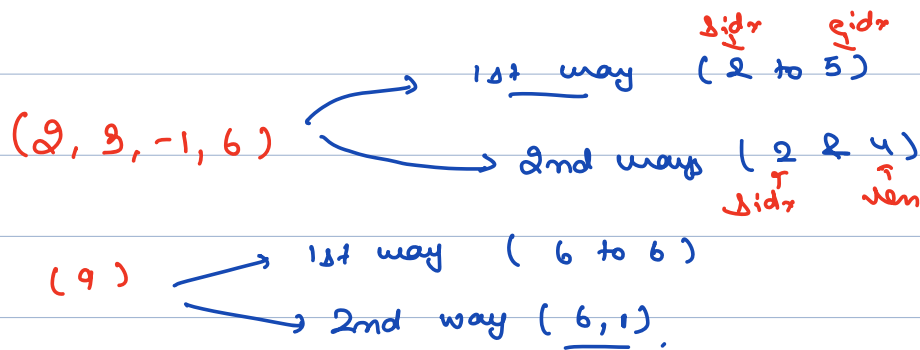
Representation of a subarray :-

→ Two ways

1) Start & end idx

2) Start Idx & len

arr[] → ^{0 1 2 3 4 5 6 7 8}
4, 1, 2, 3, -1, 6, 9, 8, 12



How many subarrays of the following array start from index 0

^{0 1 2 3 4 5 6}
[4, 2, 10, 3, 12, -2, 15]



idx

→ subarrays

(4) (0 - 0)

(4, 2) (0 - 1)

(4, 2, 10) (0 - 2)

(4, 2, 10, 3) (0 - 3)

(4, 2, 10, 3, 12) (0 - 4)

(4, 2, 10, 3, 12, -2) (0 - 5)

(4, 2, 10, 3, 12, -2, 15) (0 - 6)

How many subarrays of the following array start from index 1

^{0 1 2 3 4 5 6}
[4, 2, 10, 3, 12, -2, 15]

Subarray
(s,e)

Ans $\Rightarrow$ 6

(1,1) $\rightarrow$ 2

(1,2) $\rightarrow$ 2, 10

(1,3) $\rightarrow$ 2, 10, 3

(1,4) $\rightarrow$ 2, 10, 3, 12

(1,5) $\rightarrow$ 2, 10, 3, 12, -2

(1,6) $\rightarrow$ 2, 10, 3, 12, -2, 15

over [n]

0 1 2 3 4 . . . n-1

Subarrays starting
with 0

starting with
1

starting with 2

(0-0)

(1-1)

(2-2)

(0-1)

(1-2)

(2-3)

(0-2)

(1-3)

(2-4)

⋮

⋮

⋮

⋮

⋮

⋮

(0-(n-1))

(1-(n-1))

(2-(n-1))

n

n-1

n-2

subarrays starting with $\underline{n-1}$,

$$((n-1) - (n-1))$$

$$\frac{\quad}{1}$$

Total subarrays in an array

$$n + (n-1) + (n-2) + \dots + 2 + 1 \Rightarrow \frac{n(n+1)}{2}$$

arr $\rightarrow (1, 2, 3) \rightarrow \underline{n=3}$

subarrays $\rightarrow$
(1)

(2)

(3)

(1,2)

(2,3)

(1,2,3)

$$\frac{3(3+1)}{2} \Rightarrow \underline{6}$$

Break $\rightarrow \underline{10:15pm - 10:25pm}$

Given an array of integers and two indices, a start index and an end index, we need to print the subarrays of the array that starts from the start index and ends at the end index (both inclusive).

⁰ ¹ ² ³ ⁴
(10, 20, 30, 40, 50)

s = 2, e = 4, → (30, 40, 50)

```
void printSubarray(int arr[], int start, int end) {  
    for (int i = start; i <= end; i++) {  
        print(arr[i] << " ");  
    }  
    cout << endl;  
}
```

T.C → $O(n)$

S.C → $O(1)$.

Ques Print all subarrays of a given array.

arr = [⁰10, ¹20, ²30]

(0-0) → 10

(0-1) → 10, 20

(0-2) → 10, 20, 30

(1-1) → 20

(1-2) → 20, 30

(2-2) → 30

$$\frac{n(n+1)}{2} \Rightarrow \frac{3(3+1)}{2} = \underline{6}$$

arr = [10, 20, 30]

```
for ( s=0; s<n; s++) {
```

```
    for ( e=s; e<n; e++) {
```

```
        print (s + " " + e);
```

```
    }
}
```

s	e
0	0
0	1
0	2
1	1
1	2
2	2

(0 0) → 10

(0 1) → 10, 20

(0 2) → 10, 20, 30

(1, 1) → 20

(1, 2) → 20, 30

(2, 2) → 30

T.C → $O(n^3)$

S.C → $O(1)$.

```
for ( s=0; s<n; s++) {
```

```
    for ( e=s; e<n; e++) {
```

```
        for (i=s; i<=e; i++) {
```

```
            print(arr[i]);
```

```
        }
    }
}
```

Ques

- Given an array of N integers, return the length of smallest subarray which contains both maximum and minimum element of the array.

0 1 2 3 4 5 6 7 8 9 10
2 2 6 4 5 1 5 2 6 4 1

Max = 6
Min = 1

Ans $\rightarrow$ 3,

Brute force :-

Check all subarrays & find ans.

T.C $\rightarrow O(n^3)$

S.C $\rightarrow O(1)$.

0 1 2 3 4 5 6 7 8 9 10
2 2 6 4 5 1 5 2 6 4 1

Max = 6

min = 1

Ans $\rightarrow$ 9,

Observations :-

1) we need only 1 max & 1 min in that.

2) subarray (no max & min will be there in bins)

6
 1 1 6

max max min

min max

1 6 5 $\leftarrow$ min & max will be on the corners.

min max

max min

dry run :-

last_min_idx = ~~7~~ 8 10

last_max_idx = ~~7~~ 2 8

minValue = 1

MaxValue = 6

ans = ~~4~~ 3

idx →	0	1	2	3	4	5	6	7	8	9	10
	<u>2</u>	<u>2</u>	<u>6</u>	<u>4</u>	<u>5</u>	<u>1</u>	<u>5</u>	<u>2</u>	<u>6</u>	<u>4</u>	<u>1</u>

int min = Teda

int max = Teda

if (Max == Min) →
return 3

int last_min_idx = -1, last_max_idx = -1;

ans = Integer.MAX_VALUE;

for (i = 0; i < n; i++) {

if (arr[i] == min) {

if (last_max_idx != -1) {

ans = min(ans, i - last_max_idx + 1);

3

last_min_idx = i

3

else if (arr[i] == max) {

if (last_min_idx != -1) {

ans = min(ans, i - last_min_idx + 1);

3

last_max_idx = i;

3

T.C → O(N)

S.C → O(1)

3

Refer as;

Edge case :-

arr $\rightarrow$ 8, 8, 8 len = 1

Max = 8

Min = 8

0	1	2	3	4	5	6	7	8	9	10
<u>1</u>	<u>6</u>	<u>6</u>	<u>4</u>	<u>5</u>	<u>1</u>	<u>5</u>	<u>2</u>	<u>6</u>	<u>4</u>	<u>1</u>

min difference

11

2

100 120 130 140 | . . .